

3.2 Terraform your Lambda

 This section was helping me understand Terraform and was only here to teach me how to create a Lambda and an IAM role with Terraform; not the whole project yet.

Learning objectives

What is a provider?

The provider is just AWS. When you give Terraform access to it, it gains the ability to manage the items within that provider.

What is a resource block?

Basically every part of the infrastructure you are using — any part can be codified: “Any resource can be codified.”

What are variables and outputs for?

DEF: Terraform output variables act like return values for your infrastructure code, allowing you to expose information about your resources to the command line, external automation tools, or other Terraform modules.

What do `init`, `plan`, `apply` and `destroy` each actually do?

init = Prepares your local directory for use with Terraform by downloading required providers, retrieving external modules, and setting up your state-tracking backend.

plan = Used to see a dry run of what changes, additions, or deletions will happen.

apply = Used to execute the plan and build real-world infrastructure.

destroy = Used to delete all resources managed by your configuration.

How does your Lambda's code get packaged up and uploaded?

There are many ways to package the Lambda code. The way I have done it is by using this bit of code.

 Important: you have to run `init` EVERY time you create a new project. It just needs to initialise Terraform, SO YOU HAVE TO DO IT.

 **NOTE: Policy as code** = the idea is that as you make changes to code, you ensure they are done in a safe way. For example, are they following the company's:

- policies about cost
- security
- compliance
- etc.

The key is that as we go through the SDLC you can apply policies programmatically. In Terraform, between the plan and apply phases we take all these policies and enforce them in the middle.

plan

↓ ----- We can take the plan and inspect it and go “hey, you're about to create a protocol that turns off the encryption on the database” — that violates the security policy, so we need to change that.

apply

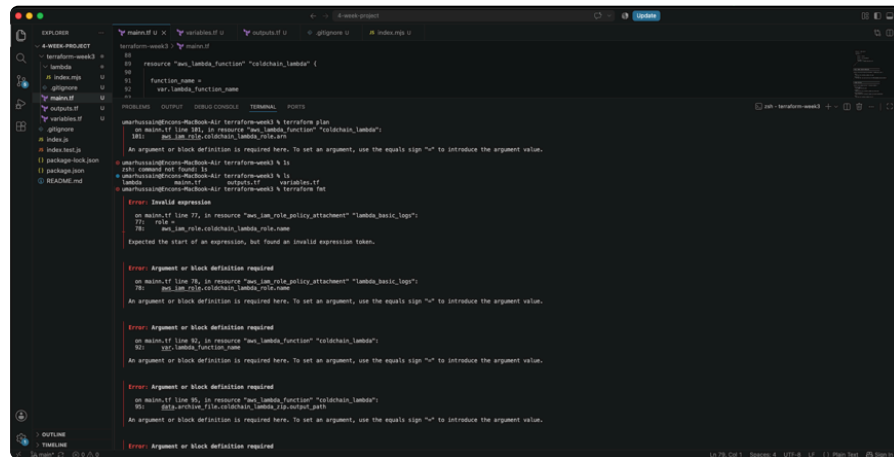
 **NOTE: Module** = in Terraform we don't want to continually write specific resource code again and again for individual resources. Instead we want to use modules:

They act as a reusable blueprint to provision standard infrastructure components across projects or environments.

So no matter what the coding language is — for example JavaScript, Python, or Ruby — modules help reuse the same patterns.

Re-setting up Terraform and AWS

Setting up Terraform



The screenshot shows a VS Code editor with a file explorer on the left and a main editor area. The file explorer shows a project structure with files like `main.tf`, `variables.tf`, `outputs.tf`, `gligione.tf`, and `index.mjs`. The main editor area displays the `main.tf` file, which contains Terraform configuration for an AWS Lambda function and its IAM role. The configuration includes resource definitions for `aws_lambda_function` and `aws_iam_role_policy_attachment`. The terminal at the bottom shows the output of the `terraform plan` command, which includes several error messages:

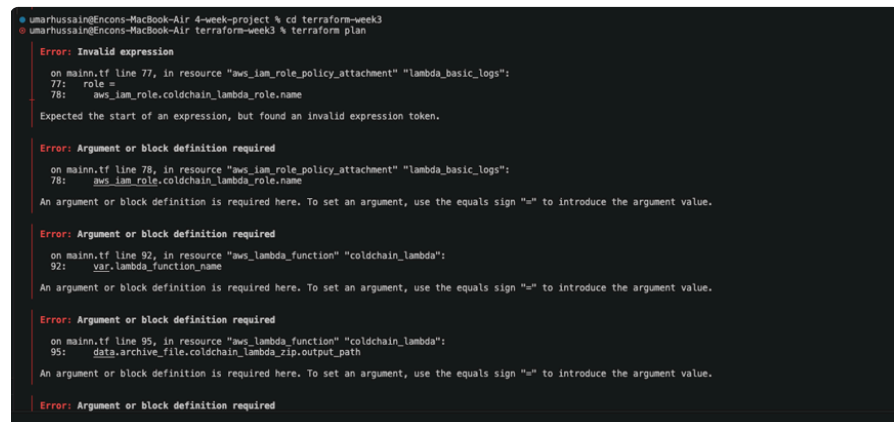
```
Error: Invalid expression
  on main.tf line 77, in resource "aws_iam_role_policy_attachment" "lambda_basic_logs":
   77:   role =
   78:     aws_iam_role.coldchain_lambda_role.name
Expected the start of an expression, but found an invalid expression token.

Error: Argument or block definition required
  on main.tf line 78, in resource "aws_iam_role_policy_attachment" "lambda_basic_logs":
   78:     aws_iam_role.coldchain_lambda_role.name
An argument or block definition is required here. To set an argument, use the equals sign "=" to introduce the argument value.

Error: Argument or block definition required
  on main.tf line 92, in resource "aws_lambda_function" "coldchain_lambda":
   92:     var.lambda_function_name
An argument or block definition is required here. To set an argument, use the equals sign "=" to introduce the argument value.

Error: Argument or block definition required
  on main.tf line 95, in resource "aws_lambda_function" "coldchain_lambda":
   95:     data.archive_file.coldchain_lambda_zip.output_path
An argument or block definition is required here. To set an argument, use the equals sign "=" to introduce the argument value.

Error: Argument or block definition required
  on main.tf line 95, in resource "aws_lambda_function" "coldchain_lambda":
   95:     data.archive_file.coldchain_lambda_zip.output_path
An argument or block definition is required here. To set an argument, use the equals sign "=" to introduce the argument value.
```



The screenshot shows a terminal window with the output of the `terraform plan` command. The output includes the same error messages as the previous screenshot:

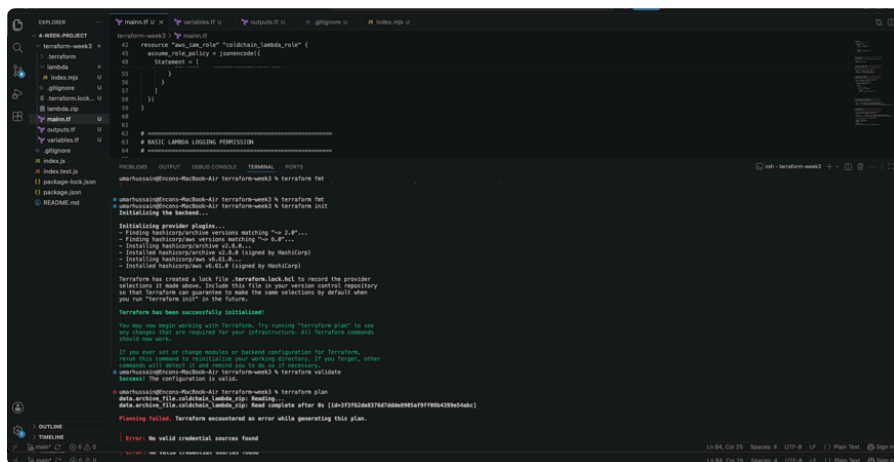
```
Error: Invalid expression
  on main.tf line 77, in resource "aws_iam_role_policy_attachment" "lambda_basic_logs":
   77:   role =
   78:     aws_iam_role.coldchain_lambda_role.name
Expected the start of an expression, but found an invalid expression token.

Error: Argument or block definition required
  on main.tf line 78, in resource "aws_iam_role_policy_attachment" "lambda_basic_logs":
   78:     aws_iam_role.coldchain_lambda_role.name
An argument or block definition is required here. To set an argument, use the equals sign "=" to introduce the argument value.

Error: Argument or block definition required
  on main.tf line 92, in resource "aws_lambda_function" "coldchain_lambda":
   92:     var.lambda_function_name
An argument or block definition is required here. To set an argument, use the equals sign "=" to introduce the argument value.

Error: Argument or block definition required
  on main.tf line 95, in resource "aws_lambda_function" "coldchain_lambda":
   95:     data.archive_file.coldchain_lambda_zip.output_path
An argument or block definition is required here. To set an argument, use the equals sign "=" to introduce the argument value.

Error: Argument or block definition required
  on main.tf line 95, in resource "aws_lambda_function" "coldchain_lambda":
   95:     data.archive_file.coldchain_lambda_zip.output_path
An argument or block definition is required here. To set an argument, use the equals sign "=" to introduce the argument value.
```



The screenshot shows a VS Code editor with a file explorer on the left and a main editor area. The file explorer shows a project structure with files like `main.tf`, `variables.tf`, `outputs.tf`, `gligione.tf`, and `index.mjs`. The main editor area displays the `main.tf` file, which contains Terraform configuration for an AWS Lambda function and its IAM role. The terminal at the bottom shows the output of the `terraform plan` command, which includes the same error messages as the previous screenshot:

```
Error: Invalid expression
  on main.tf line 77, in resource "aws_iam_role_policy_attachment" "lambda_basic_logs":
   77:   role =
   78:     aws_iam_role.coldchain_lambda_role.name
Expected the start of an expression, but found an invalid expression token.

Error: Argument or block definition required
  on main.tf line 78, in resource "aws_iam_role_policy_attachment" "lambda_basic_logs":
   78:     aws_iam_role.coldchain_lambda_role.name
An argument or block definition is required here. To set an argument, use the equals sign "=" to introduce the argument value.

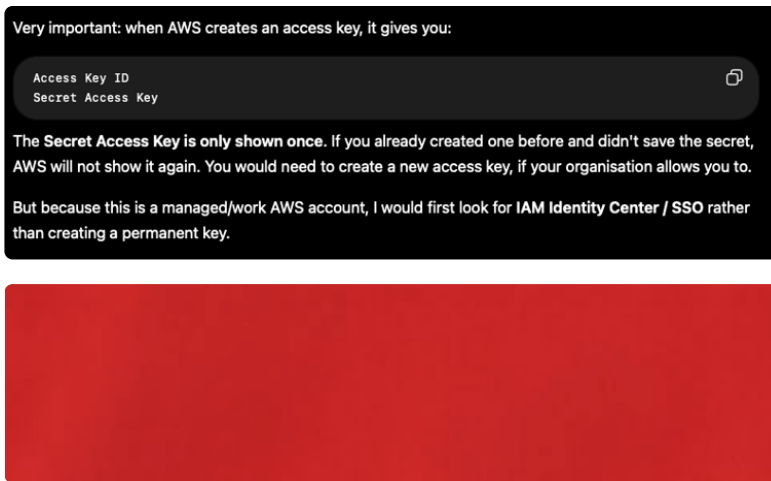
Error: Argument or block definition required
  on main.tf line 92, in resource "aws_lambda_function" "coldchain_lambda":
   92:     var.lambda_function_name
An argument or block definition is required here. To set an argument, use the equals sign "=" to introduce the argument value.

Error: Argument or block definition required
  on main.tf line 95, in resource "aws_lambda_function" "coldchain_lambda":
   95:     data.archive_file.coldchain_lambda_zip.output_path
An argument or block definition is required here. To set an argument, use the equals sign "=" to introduce the argument value.

Error: Argument or block definition required
  on main.tf line 95, in resource "aws_lambda_function" "coldchain_lambda":
   95:     data.archive_file.coldchain_lambda_zip.output_path
An argument or block definition is required here. To set an argument, use the equals sign "=" to introduce the argument value.
```

(I have pasted these errors well after I actually did this task, so I can't even remember what they were about or what the context was behind these errors. So future me, if you are confused, know that only Allah knows what the hell the context behind these errors was).

Giving AWS access keys



(UPDATE: there was a screenshot of the access keys where the red boxes are, but I was told to delete it because it had my access keys in it — a big security risk.) The screenshot above shows me giving global access from my AWS account to my computer. After running `aws configure` and inputting both the access and secret keys (which I got from the IAM user's security credentials) and the correct region, my computer now had access to my AWS account.

Task

- Start from an empty folder.
- Write Terraform that creates your Lambda function and the IAM role it needs.
- `{{terraform apply}}`, then check it in the console.
- `{{terraform destroy}}`, then check it's gone.
- `{{terraform apply}}` again.

To start with, in the same git repo folder as the first 2 weeks of the project, I created a new folder:

```
1 terraform-week3/  
2 |  
3 |— main.tf  
4 |— variables.tf  
5 |— outputs.tf  
6 |— .gitignore  
7 |  
8 |— lambda/  
9 |   |— index.mjs
```

[main.tf](#) = says things like:

Create an IAM role.
Create a Lambda.
Give the Lambda this role.

`index.mjs` = this holds the actual JavaScript that runs **inside** the Lambda.

[Terraform](#) | [HashiCorp Developer](#) and [output.tf](#) = are just ways of keeping your Terraform project organised.

`variables.tf` is where you define values that Terraform can reuse. Instead of hard-coding the same value again and again inside `main.tf`, you give it a variable name.

For example:

```
1 variable "aws_region" {
```

```

2   type    = string
3   default = "eu-north-1"
4 }

```

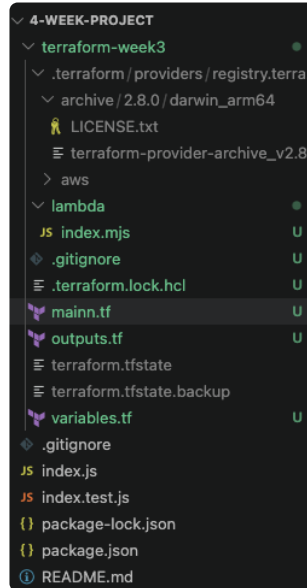
Then inside `main.tf` you can write:

```

1 provider "aws" {
2   region = var.aws_region
3 }

```

This is what my files looked like:



Then used AI to create the Terraform code but here is what I learned:

```

data "archive_file" "coldchain_lambda_zip" {
  type      = "zip"
  source_dir = "${path.module}/lambda"
  output_path = "${path.module}/lambda.zip"
}

```

The code above will pack the lambda index.mjs code into a zip file with the rest of the configuration and then send it across.

The output folder will hold the outputs of the plan — that is the latest plan. This is the plan that your Terraform will apply, has applied, and will compare against other things too.

Below is the code that will go at the top of the [main.tf](#) document

```

1 terraform {
2   required_version = ">= 1.2.0"
3
4   required_providers {
5     aws = {
6       source = "hashicorp/aws"
7       version = "~> 6.0"
8     }
9   }
10  archive = {
11    source = "hashicorp/archive"
12    version = "~> 2.0"
13  }
14 }

```

This will tell the files what Terraform versions we are using, the provider (AWS), and that the archive provider is available.

Then I ran terraform plan

[illegible]

Then I ran terraform apply

[illegible]

Then I ran terraform destroy

[illegible]

